APM52009 – Machine Learning for Scientific Computing and Numerical Analysis

Week 2: Neural Network Approximation Theory

Victorita Dolean and Hadrien Montanelli

Abstract

In this chapter, we address the approximation error discussed in the first chapter by examining the foundations of neural network approximation. At its core is the universality theorem, which states that neural networks with a single hidden layer can approximate any continuous function on compact sets, provided they have enough neurons. We will also look at recent advances that quantify the approximation capabilities of deeper networks, particularly the work of Yarotsky (2017), which provides near-optimal bounds on the approximation rates of deep ReLU networks for functions with a finite number of bounded derivatives. Finally, we draw connections with finite elements, revealing a structural equivalence between neural and finite element spaces: ReLU and ReLU^k activations correspond to piecewise-polynomial bases and their hierarchical refinements.

Contents

1 Artificial neural networks **2**

1.1 Algebraic objects . . . 2

1.2 Geometric objects . . . 3

2 Universality theorems **4**

3 Approximation in Sobolev spaces **6**

3.1 Constructive blocks: from sawtooth to multiplication . . . 6

3.2 Convergence rates in Sobolev spaces . . . 9

3.3 Discussion . . . 10

3.4 Breaking the curse of dimensionality . . . 10

4 Structural equivalence with finite elements **12**

5 Exercises **14**

6 Suggested reading **15**

1 Artificial neural networks

Neural networks are *parametric families of functions*. They can be fruitfully viewed as:

- **Algebraic objects:** compositions of linear operators and nonlinear activations;
- **Geometric objects:** piecewise-smooth manifolds in $\mathbb{R}^d$ with data-adaptive breakpoints;
- **Numerical analogues:** adaptive finite-element or spline spaces, where *depth* plays the role of hierarchy and *width* controls local resolution (see Section 4).

1.1 Algebraic objects

The fundamental unit of an artificial neural network is the *artificial neuron*, a concept that dates back to the model of McCulloch and Pitts (1943) [7].

Definition 1.1 (Artificial neuron)**.** Let d_0 and d_1 be two positive integers. An artificial neuron with weights $W \in \mathbb{R}^{d_1 \times d_0}$, bias $\boldsymbol{b} \in \mathbb{R}^{d_1}$, and activation function $\sigma : \mathbb{R}^{d_1} \to \mathbb{R}^{d_1}$ is the function

$$\boldsymbol{a} : \mathbb{R}^{d_0} \to \mathbb{R}^{d_1}, \qquad \boldsymbol{a}(\boldsymbol{x}) = \sigma\left(W\boldsymbol{x} + \boldsymbol{b}\right).$$

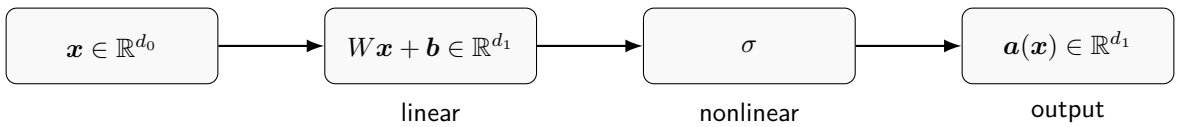


Figure 1: Computation in one neuron: affine map followed by a nonlinear activation.

By composing many such neurons, one obtains an *artificial neural network*.

Definition 1.2 (Artificial neural network)**.** Let $d_0, \ldots, d_{L+1}$ be positive integers such that $d_0 = d$ and $d_{L+1} = 1$. A feedforward neural network is the composition

$$f_\theta : \mathbb{R}^d \to \mathbb{R}, \qquad f_\theta = a^{(L+1)} \circ \boldsymbol{a}^{(L)} \circ \cdots \circ \boldsymbol{a}^{(0)},$$

with

$$\begin{aligned}
&\text{(input layer)} && \boldsymbol{a}^{(0)} : \mathbb{R}^d \to \mathbb{R}^d, && \boldsymbol{a}^{(0)}(\boldsymbol{x}) = \boldsymbol{x}, \\
&\text{(hidden layers)} && \boldsymbol{a}^{(\ell)} : \mathbb{R}^{d_{\ell-1}} \to \mathbb{R}^{d_\ell}, && \boldsymbol{a}^{(\ell)}(\boldsymbol{x}) = \sigma\left(W^{(\ell-1)}\boldsymbol{x} + \boldsymbol{b}^{(\ell-1)}\right), \quad 1 \le \ell \le L, \\
&\text{(output layer)} && a^{(L+1)} : \mathbb{R}^{d_L} \to \mathbb{R}, && a^{(L+1)}(\boldsymbol{x}) = W^{(L)}\boldsymbol{x} + b^{(L)},
\end{aligned}$$

where $W^{(\ell-1)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, $\boldsymbol{b}^{(\ell-1)} \in \mathbb{R}^{d_\ell}$, $W^{(L)} \in \mathbb{R}^{1 \times d_L}$, and $b^{(L)} \in \mathbb{R}$. We set

$$\boldsymbol{\theta} = \left((W^{(0)}, \boldsymbol{b}^{(0)}), \ldots, (W^{(L)}, b^{(L)})\right).$$

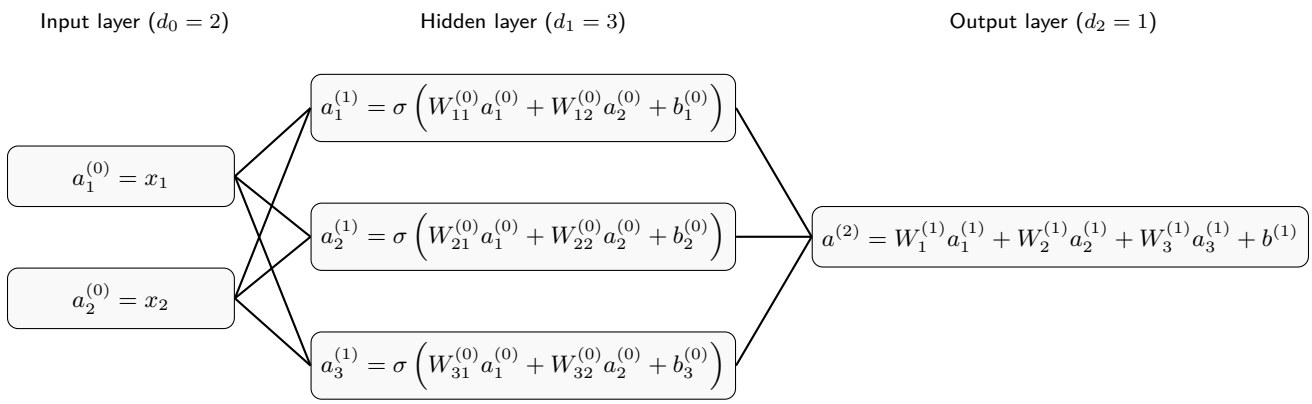


Figure 2: Feedforward network $f_\theta = a^{(2)} \circ \boldsymbol{a}^{(1)} \circ \boldsymbol{a}^{(0)}$ with one hidden layer ($L = 1$), mapping $\mathbb{R}^2$ to $\mathbb{R}$.

The evaluation of a neural network is called a *forward pass*.

Algorithm 1 Forward pass of a neural network

Require: Input vector $\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^d$

for $\ell = 1, 2, \ldots, L$ **do**

  Linear step: $\boldsymbol{z}^{(\ell-1)} = W^{(\ell-1)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell-1)}$

  Nonlinear step: $\boldsymbol{a}^{(\ell)} = \sigma\big(\boldsymbol{z}^{(\ell-1)}\big)$

end for

Output: $a^{(L+1)} = W^{(L)}\boldsymbol{a}^{(L)} + b^{(L)}$

We now introduce a few notions to characterize the structure of such a network.

Definition 1.3 (Depth, width, and size)**.** We call L the *depth* (number of hidden layers) and define

$$\text{width} = \max_{\ell=0,\ldots,L} d_\ell, \qquad \text{size} = \Big|\{(i,j,\ell) : W_{i,j}^{(\ell)} \neq 0\} \cup \{(i,\ell) : \boldsymbol{b}_i^{(\ell)} \neq 0\}\Big| \;\leq\; p = \sum_{\ell=1}^{L+1}(d_\ell d_{\ell-1} + d_\ell).$$

Finally, we recall some standard activation functions commonly used in neural networks:

$$\sigma(x) = \frac{1}{1+e^{-x}} \quad \text{(sigmoid)}, \qquad \sigma(x) = \tanh(x), \qquad \sigma(x) = \max(0,x) \quad \text{(ReLU)}.$$

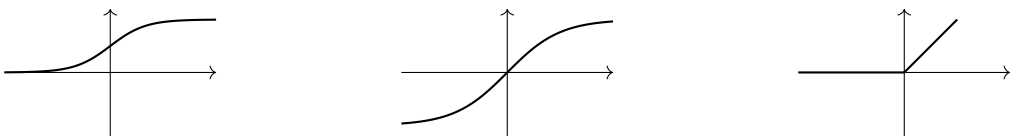


Figure 3: Standard activation functions, acting componentwise.

Remark 1.4. Linear regression can be viewed as a depth-0 network (no hidden layer, only an output layer), whereas nonlinear regression corresponds to a depth-1 network with no separate output layer.

1.2 Geometric objects

A ReLU network partitions $\mathbb{R}^d$ into convex polytopes determined by the sign of $W^{(\ell-1)}\boldsymbol{x} + \boldsymbol{b}^{(\ell-1)}$. On each region, it acts affinely, yielding a piecewise-linear approximation [4, Lem. 2.1].

Lemma 1.5. *Every ReLU network $f_\theta : \mathbb{R}^d \to \mathbb{R}$ is continuous and piecewise linear. Equivalently, there exists a polyhedral decomposition of $\mathbb{R}^d$ such that f_θ is affine on each cell.*

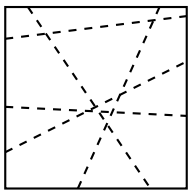


Figure 4: ReLU networks partition $\mathbb{R}^2$ into affine regions separated by learned hyperplanes.

Takeaways

- **Algebra:** Compositions of affine maps and nonlinearities parameterized by weights.
- **Depth vs. width:** Depth controls composition; width sets the number of affine regions.
- **Geometry:** ReLU networks define adaptive polyhedral partitions where they act affinely.

2 Universality theorems

In Section 1, we described neural networks as compositions of affine maps and nonlinearities, with a geometric interpretation as piecewise-linear functions defined on polyhedral partitions. A natural question follows: *Given a target function f on a compact domain $D \subset \mathbb{R}^d$, can such models approximate f arbitrarily well?*

This question of *universality* predates neural networks: for polynomials, it is settled by the classical Stone–Weierstrass theorem. Modern results extend this principle to shallow neural networks with suitable activations, establishing that they too can approximate any continuous function. This section formalizes these results and clarifies their scope.

Universality for polynomials It is known since Weierstrass in 1885 that polynomials with real coefficients are dense in the space of real-valued, continuous functions in one dimension. It was extended by Stone in 1937 to more general cases in higher dimensions. This is the Stone–Weierstrass theorem. Let $C(D)$ denote the space of continuous functions in $D \subset \mathbb{R}^d$.

Theorem 2.1 (Stone–Weierstrass)**.** *Let $D \subset \mathbb{R}^d$ be compact and $F \subset C(D)$ satisfy:*

(i) for all $\boldsymbol{x} \in D$, there exists $f \in F$ such that $f(\boldsymbol{x}) \neq 0$;

(ii) for all $\boldsymbol{x} \neq \boldsymbol{y} \in D$, there exists $f \in F$ such that $f(\boldsymbol{x}) \neq f(\boldsymbol{y})$;

(iii) F is a subalgebra (closed under addition, multiplication, scalar multiplication).

Then F is dense in $C(D)$ (with the uniform norm).

Polynomials satisfy these assumptions. Neural networks *do not* form a subalgebra (they are not closed under multiplication), which is why Stone–Weierstrass does not apply directly; their universality must be proved by different means.

Universality for sigmoids Universality theorems for neural networks date back to Cybenko [2, Thm. 2] and Hornik et al. [5, Thm. 2.1], both in 1989. The next theorem is Cybenko's.

Theorem 2.2 (Universality for sigmoids)**.** *Let σ be continuous and sigmoidal, i.e., $\sigma(t) \to 0$ as $t \to -\infty$ and $\sigma(t) \to 1$ as $t \to +\infty$. Then finite sums*

$$G_N(\boldsymbol{x}) = \sum_{j=1}^{N} \alpha_j \, \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)$$

are dense in $C([0,1]^d)$. In other words, given any $f \in C([0,1]^d)$ and $\epsilon > 0$, there exist $N \geq 1$ and parameters such that $|G_N(\boldsymbol{x}) - f(\boldsymbol{x})| < \epsilon$ for all $\boldsymbol{x}$ in $[0,1]$.

This is a depth-one (shallow) network $W^{(2)}\sigma(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$ of width $\max\{d, N\}$ and size $(d+2)N$.

Universality beyond sigmoids Leshno, Lin, Pinkus, and Schocken extended in 1993 universality to a broad class of activations: it is enough that σ be *nonpolynomial* [6, Thm. 1].

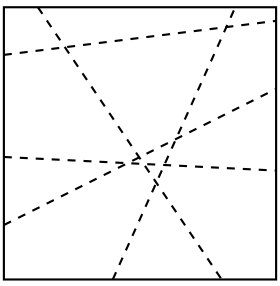
Theorem 2.3 (Universality for nonpolynomial activations)**.** *Let σ be locally bounded and piecewise continuous. Then finite sums*

$$G_N(\boldsymbol{x}) = \sum_{j=1}^{N} \alpha_j \, \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)$$

are dense in $C([0,1]^d)$ if and only if σ is not a polynomial.

Many additional variants and proofs are surveyed in Pinkus (1999) [12].

Intuition behind universality Each hidden unit $\sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$ "bends" the function along the hyperplane $\{\boldsymbol{x} : \boldsymbol{w}^\top \boldsymbol{x} + b = 0\}$. With enough units, the domain is partitioned into small cells on which G_N is nearly constant or affine, allowing uniform approximation.



many hyperplanes $\Rightarrow$ fine partition $\Rightarrow$ uniform approximation

Figure 5: Shallow nets split the domain into regions with simple output, enabling uniform approximation.

Another view—a shallow network approximates a function as a linear combination of *ridge functions*:

$$G_N(\boldsymbol{x}) = \sum_{j=1}^{N} \alpha_j \, \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j),$$

where the hidden layer provides a flexible dictionary $\{\sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)\}$ and the output layer selects appropriate coefficients α_j. The ridge functions are linearly independent under mild conditions:

Theorem 2.4 (Linear independence of ridge functions)**.** *The functions $\{\sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)\}_{j=1}^N$ are linearly independent if the parameter pairs $(\boldsymbol{w}_i, b_i)$ are linearly independent in $\mathbb{R}^{d+1}$ for $i \neq j$.*

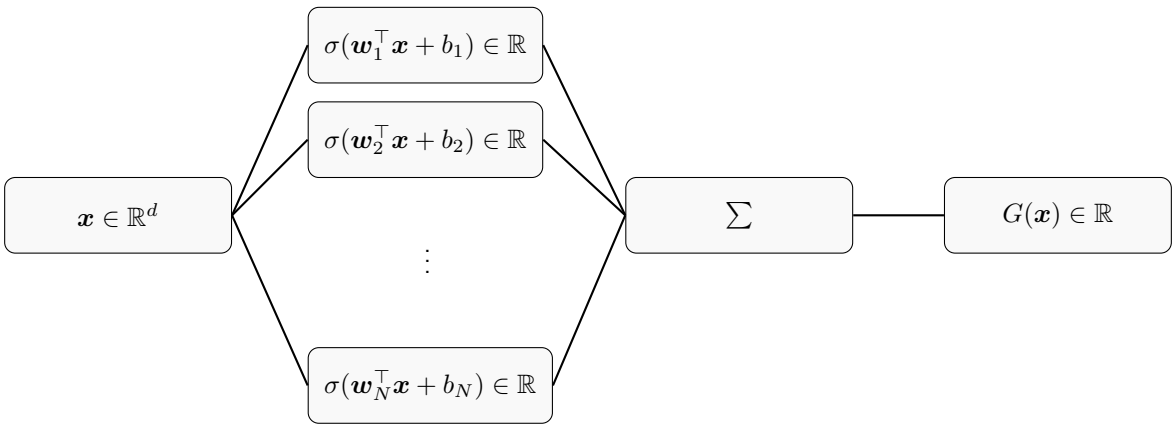


Figure 6: Shallow network as a *ridge-function dictionary* combined linearly. Each hidden neuron provides a ridge function, and the output layer forms a linear combination.

Takeaways

Universality of feedforward neural networks conveys several insights but has clear limitations:

- **Universal approximation:** Networks with enough neurons can approximate any continuous function on a compact domain in principle.
- **Conceptual significance:** Shows neural networks are flexible function classes and provides theoretical justification for their expressive power.
- **Non-constructive:** Does not specify required width, depth, or explicit weights.
- **Scope:** Qualitative statement only; does not quantify scaling of N with input dimension d, error tolerance ϵ, smoothness, or enforce symmetries, conservation, or stability.

3 Approximation in Sobolev spaces

The universality theorem is *qualitative*: it ensures neural networks can approximate any continuous function but gives no rate. We now turn to *quantitative* results, first established by Yarotsky (2017) [14], relating network size to function smoothness, accuracy, and input dimension.

Curse of dimensionality Consider a function $f : D \subset \mathbb{R}^d \to \mathbb{R}$ of smoothness s, and let $\epsilon \in (0,1)$. Many approximation results aim to construct a network f_N of size N satisfying

$$\|f - f_N\| \leq \epsilon, \qquad N = \mathcal{O}(\epsilon^{-d/s}). \tag{3.1}$$

Such estimates are typical in Sobolev-type settings. However, these *Sobolev-like* rates reveal the *curse of dimensionality*: the network size grows exponentially with the dimension d.

Definition 3.1 (Sobolev spaces)**.** The Sobolev space $W^{s,q}(D)$, $s \in \mathbb{N}$, $1 \leq q \leq \infty$, is defined by

$$W^{s,q}(D) = \{f \in L^q(D) : \partial^{\boldsymbol{k}} f \in L^q(D) \text{ for all } |\boldsymbol{k}|_1 \leq s\},$$

where $\boldsymbol{k} = (k_1, \ldots, k_d) \in \mathbb{N}^d$ and $\partial^{\boldsymbol{k}} f = \partial_{x_1}^{k_1} \cdots \partial_{x_d}^{k_d} f$. Equipped with the norm

$$\|f\|_{s,q} = \begin{cases} \left(\sum_{|\boldsymbol{k}|_1 \leq s} \|\partial^{\boldsymbol{k}} f\|_q^q\right)^{1/q}, & q < \infty, \\ \max_{|\boldsymbol{k}|_1 \leq s} \|\partial^{\boldsymbol{k}} f\|_\infty, & q = \infty, \end{cases}$$

these Banach spaces are the *completion* of $C^s(D)$; for $q = 2$, $H^s(D) = W^{s,2}(D)$.

3.1 Constructive blocks: from sawtooth to multiplication

Yarotsky's constructive proof shows that *deep ReLU* networks can emulate products and polynomials with *logarithmic depth*, achieving near-optimal Sobolev rates (matching Eq. (3.1) up to logs) with $\sigma(x) = \mathrm{ReLU}(x)$.

Let $g : [0,1] \to [0,1]$ be defined by $g(x) = 1 - |1 - 2x| = 2\sigma(x) - 4\sigma(x - 1/2)$. We note that $g(x/2) = x$ and $g(1-x) = g(x)$ for all $x \in [0,1]$. We then define $g_1(x) = g(x)$ and $g_n(x) = g \circ g_{n-1}(x)$ for $n \geq 2$.

Lemma 3.2 (Properties of g_n)**.** *For all $n \geq 1$,*

$$g_n(x) = 2^n x, \quad 0 \leq x \leq \frac{1}{2^n}, \qquad g_n(x) = g_n\left(\frac{1}{2^{n-1}} - x\right), \quad \frac{1}{2^n} \leq x \leq \frac{1}{2^{n-1}}.$$

For all $n \geq 2$, g_n is $1/2^{n-1}$-periodic on $[0,1]$, that is,

$$g_n\left(x + \frac{1}{2^{n-1}}\right) = g_n(x), \quad 0 \leq x \leq \frac{1}{2^{n-1}}.$$

Proof

We start with the first property. For $n = 1$, $g_1(x) = g(x) = 2x$ on $[0, 1/2]$. For any $n \geq 2$, we proceed by induction. Let $x \in [0, 1/2^n]$. Then

$$g_n(x) = g \circ g_{n-1}(x) = g(2^{n-1}x) = 2^n x,$$

using the induction hypothesis for $x \in [0, 1/2^n] \subset [0, 1/2^{n-1}]$ and $g(x) = 2x$ on $[0, 1/2^n] \subset [0, 1/2]$. Let $n \geq 1$ and $x \in [1/2^n, 1/2^{n-1}]$. To prove the second property, we write

$$g_n(x) = g(g_{n-1}(x)) = g(2^{n-1}x) = g(1 - 2^{n-1}x) = g\left(g_{n-1}\left(\frac{1}{2^{n-1}} - x\right)\right) = g_n\left(\frac{1}{2^{n-1}} - x\right).$$

For periodicity, for $n = 2$, take $x \in [0, 1/2]$. Then $g_2(x + 1/2) = g_2(1/2 - (x + 1/2)) = g_2(x)$. Take $n \geq 2$ and $0 \leq x \leq 1/2^{n-1} \leq 1/2^{n-2}$. Then

$$g_n\left(x + \frac{1}{2^{n-1}}\right) = g_2\left(g_{n-2}\left(x + \frac{1}{2^{n-2}}\right)\right) = g_2\left(2^{n-2}\left(x + \frac{1}{2^{n-1}}\right)\right) = g_2(2^{n-2}x) = g_n(x).$$

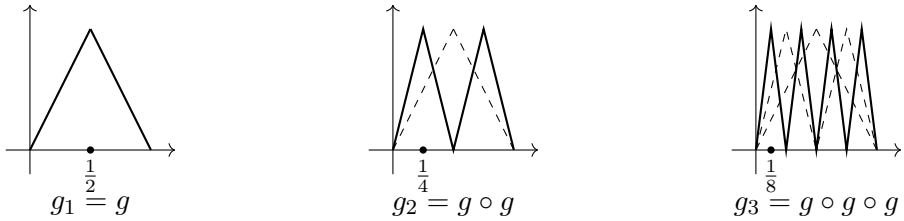


Figure 7: Iterated sawtooth maps g_n for $n = 1, 2, 3$: each iteration doubles the number of linear pieces.

Lemma 3.3 (Series representation of x^2)**.** *For any $x \in [0,1]$,*

$$x^2 = x - \sum_{n \geq 1} \frac{g_n(x)}{4^n}.$$

Proof

Set, for any $x \in [0,1]$,

$$H(x) = x - \sum_{n \geq 1} \frac{g_n(x)}{4^n} = \sum_{n \geq 1} \frac{h_n(x)}{4^n}, \tag{3.2}$$

where $h_n(x) = 2^n - g_n(x)$. By construction, for $n \geq 2$, $h_n(x/2) = h_{n-1}(x)$ and $h_1(x/2) = 0$. Hence

$$H\left(\frac{x}{2}\right) = \sum_{n \geq 2} \frac{h_{n-1}(x)}{4^n} = \frac{H(x)}{4}. \tag{3.3}$$

We also have

$$H(1-x) = 1 - x - \sum_{n \geq 1} \frac{g_n(1-x)}{4^n} = 1 - x - \sum_{n \geq 1} \frac{g_n(x)}{4^n} = H(x) + 1 - 2x. \tag{3.4}$$

We leave as an exercise that the former two properties imply that

$$H\left(\frac{k}{2^n}\right) = \left(\frac{k}{2^n}\right)^2, \quad 0 \leq k \leq 2^n, \quad n \geq 0. \tag{3.5}$$

This means that $H(x) = x^2$ on a dense subset of $[0,1]$. We must show that H is continuous to conclude. In fact it is Lipschitz continuous since g_n is Lipschitz with constant 2^n and

$$|H(x) - H(y)| \leq |x - y| + \sum_{n \geq 1} \frac{2^n}{4^n} |x - y| \leq 2|x - y| \qquad \forall x, y \in [0,1]. \tag{3.6}$$

Lemma 3.4 (Approximating x^2)**.** *For any $\epsilon \in (0,1)$, the function $f(x) = x^2$ on $[0,1]$ can be approximated with error ϵ by a ReLU network that has depth and size $\mathcal{O}(\log(1/\epsilon))$.*

Proof

Let $\hat{f}_n$ be the ReLU network

$$\hat{f}_n(x) = x - \sum_{j=1}^{n} \frac{g_j(x)}{4^j}, \quad x \in [0,1]. \tag{3.7}$$

It has depth and size $\mathcal{O}(n)$. For any $x \in [0,1]$, we have

$$|f(x) - \hat{f}_n(x)| \leq \sum_{j \geq n+1} \frac{1}{4^j} = \frac{1}{3} \frac{1}{4^n}. \tag{3.8}$$

We take $n = \mathcal{O}(\log(1/\epsilon))$ to conclude.

Lemma 3.4 shows that ReLU networks can efficiently approximate quadratic nonlinearities, with exponentially decaying error as the depth grows; equivalently, achieving accuracy ϵ requires only depth and size $\mathcal{O}(\log(1/\epsilon))$.

The next step is to leverage this building block to construct a network that performs *multiplication* between two inputs. It relies on the polarization identity $xy = \frac{1}{4}((x+y)^2 - (x-y)^2)$.

Lemma 3.5 (Multiplication of two inputs)**.** *For any $\epsilon \in (0,1)$, there is a deep ReLU network $\widehat{\Pi}$ that has depth $\mathcal{O}(\log(1/\epsilon))$ and size $\mathcal{O}(\log(1/\epsilon))$ such that*

$$|xy - \widehat{\Pi}(x,y)| \leq \epsilon \qquad \forall x, y \in [-1,1].$$

Moreover, if $x = 0$ or $y = 0$, then $\widehat{\Pi}(x,y) = 0$.

Proof

Let $x, y \in [-1,1]$. Define the network as

$$\widehat{\Pi}(x,y) = \hat{f}_n\left(\frac{\sigma(x+y)}{2} + \frac{\sigma(-x-y)}{2}\right) - \hat{f}_n\left(\frac{\sigma(x-y)}{2} + \frac{\sigma(y-x)}{2}\right), \tag{3.9}$$

with $\hat{f}_n$ as in Lemma 3.4. It has depth and size $\mathcal{O}(n)$. Since $\sigma(x) = \max(0,x) = (x+|x|)/2$, we have

$$\sigma(x+y) + \sigma(-x-y) = |x+y|, \quad \sigma(x-y) + \sigma(y-x) = |x-y|. \tag{3.10}$$

This yields

$$\widehat{\Pi}(x,y) = \hat{f}_n\left(\frac{|x+y|}{2}\right) - \hat{f}_n\left(\frac{|x-y|}{2}\right). \tag{3.11}$$

Therefore,

$$|xy - \widehat{\Pi}(x,y)| = \left|\frac{4(\frac{x+y}{2})^2 - 4(\frac{x-y}{2})^2}{4} - \frac{4\hat{f}_n\left(\frac{|x+y|}{2}\right) - 4\hat{f}_n\left(\frac{|x-y|}{2}\right)}{4}\right| \leq \frac{2}{3}\frac{1}{4^n}. \tag{3.12}$$

We used Lemma 3.4 with $|x+y|/2$ and $|x-y|/2$ in $[0,1]$. We take $n = \mathcal{O}(\log(1/\epsilon))$ to conclude.

Lemma 3.5 shows that ReLU networks can reproduce multiplication with logarithmic depth and size. This was first generalized to any dimension and input length in [10, Prop. 2.2]; see also [11, Prop. 7.4].

Lemma 3.6 (Multiplication of many inputs)**.** *For any $d \geq 2$, $M \geq 1$ and $\epsilon \in (0,1)$, there is a deep ReLU network $\widehat{\Pi}$ that has depth $\mathcal{O}(\log d \log(d/\epsilon))$ and size $\mathcal{O}(d \log(d/\epsilon))$ such that*

$$|\Pi_{j=1}^d x_j - \widehat{\Pi}(x_1, \ldots, x_d)| \leq \epsilon \qquad \forall x_1, \ldots, x_d \in [-M, M].$$

Moreover, if any of the input $x_j = 0$, then $\widehat{\Pi}(x_1, \ldots, x_d) = 0$.

Since we can now approximate multiplication and hence any polynomial efficiently, classical approximation results follow from combining these constructions with standard polynomial approximation theory. For example:

- **Univariate:** Chebyshev or Taylor polynomials allow any smooth $f : [0,1] \to \mathbb{R}$ to be approximated by p_n of degree n; a ReLU network then implements p_n with depth and size $\mathcal{O}(\log n)$.
- **Multivariate:** Tensor-product or sparse-grid polynomials in d dimensions can be implemented via repeated multiplications, giving network size scaling polynomially with n and d.
- **Sobolev/Korobov/Hölder classes:** Known polynomial approximation rates yield quantitative bounds on the ReLU network size for a given error.

Equivalently, any classical polynomial approximation result immediately gives a corresponding ReLU network with controlled depth and size.

3.2 Convergence rates in Sobolev spaces

To prove the approximation results one uses the decomposition of the approximation error

$$\|f - \hat{f}_N\| \leq \|f - f_N\| + \|f_N - \hat{f}_N\|,$$

for some norm $\|\cdot\|$. Yarotsky utilizes sum-products of local Taylor polynomials [14, Thm. 1].

Let $B^{s,\infty}([0,1]^d)$ be the unit ball in $W^{s,\infty}([0,1]^d)$, i.e., $B^{s,\infty}([0,1]^d) = \{f \in W^{s,\infty}([0,1]^d) : \|f\|_{s,\infty} \leq 1\}$.

Theorem 3.7 (Error bounds in Sobolev spaces)**.** *For any $d, s \geq 1$, $\epsilon \in (0,1)$, and f in $B^{s,\infty}([0,1]^d)$, there is a ReLU network $\hat{f}$ with depth $\mathcal{O}(\log(1/\epsilon))$ and size $\mathcal{O}(\epsilon^{-d/s}\log(1/\epsilon))$ such that*

$$\|f - \hat{f}\|_\infty \leq \epsilon.$$

Proof

We prove the 1D case; higher dimensions are similar but slightly more technical. Consider

$$f_L(x) = \sum_{\ell=0}^{L} \phi_\ell(x) P_\ell(x), \quad L \geq 1,$$

with

$$\phi_\ell(x) = \psi\Big(3L\Big(x - \frac{\ell}{L}\Big)\Big), \quad \psi(x) = \sigma(x+2) + \sigma(x-2) - \sigma(x+1) - \sigma(x-1),$$

and

$$P_\ell(x) = \sum_{j=0}^{s-1} \frac{f^{(j)}(\frac{\ell}{L})}{j!}\Big(x - \frac{\ell}{L}\Big)^j.$$

The ϕ_ℓ's satisfy $\operatorname{supp}\phi_\ell = \{x : |x - \ell/L| < 1/L\}$ and form a partition of unity.
First, if $L = \mathcal{O}(\epsilon^{-\frac{1}{s}})$, then $\|f - f_L\|_\infty \leq C\epsilon$ for some constant $C > 0$ independent of ϵ. Indeed,

$$|f(x) - f_L(x)| \leq \sum_{\ell=0}^{L} |\phi_\ell(x)||f(x) - P_\ell(x)| \leq \sum_{\ell:|x-\ell/L|<1/L} |f(x) - P_\ell(x)| \leq 2\frac{1}{s!}\left(\frac{1}{L}\right)^s \leq C\epsilon.$$

Next, we approximate f_L by a deep network. Define

$$\hat{f}_L(x) = \sum_{\ell=0}^{L} \widehat{\Pi}(\phi_l(x), \widehat{P}_l(x)), \quad \widehat{P}_\ell(x) = \sum_{j=0}^{s-1} \frac{f^{(j)}(\frac{\ell}{L})}{j!}\widehat{\Pi}\Big(\underbrace{x - \frac{\ell}{L}, \ldots, x - \frac{\ell}{L}}_{j \text{ times}}\Big).$$

Then

$$|P_\ell(x) - \widehat{P}_\ell(x)| \leq \sum_{j=0}^{s-1} \left|\frac{f^{(j)}(\frac{\ell}{L})}{j!}\right| \left|\Big(x - \frac{\ell}{L}\Big)^j - \widehat{\Pi}\Big(x - \frac{\ell}{L}, \ldots, x - \frac{\ell}{L}\Big)\right| \leq C\epsilon,$$

whenever $\widehat{P}_\ell$ has depth $\mathcal{O}(\log s \log(s/\epsilon)) = \mathcal{O}(\log(1/\epsilon))$ and size $\mathcal{O}(s\log(s/\epsilon)) = \mathcal{O}(\log(1/\epsilon))$ using Lemma 3.6. Moreover,

$$|f_L(x) - \hat{f}_L(x)| \leq \sum_{\ell:|x-\ell/L|<1/L} \Big(|\phi_\ell(x)||P_\ell(x) - \widehat{P}_\ell(x)| + |\phi_\ell(x)\widehat{P}_\ell(x) - \widehat{\Pi}(\phi_\ell(x), \widehat{P}_\ell(x))|\Big) \leq C\epsilon,$$

whenever $\widehat{\Pi}$ has depth and size $\mathcal{O}(\log(1/\epsilon))$. The resulting network has depth

$$\operatorname{depth}(\widehat{\Pi}) + \max_{0\leq\ell\leq L} \operatorname{depth}(\widehat{P}_\ell) \leq C\log(1/\epsilon)$$

and its size is bounded by

$$C\sum_{\ell=0}^{L}\Big(\operatorname{size}(\widehat{\Pi}) + \operatorname{size}(\widehat{P}_\ell)\Big) \leq CL\log(1/\epsilon).$$

Since $L = \mathcal{O}(\epsilon^{-\frac{1}{s}})$, the result follows.

3.3 Discussion

The constructive Sobolev approach provides explicit, deterministic bounds by building deep ReLU networks that emulate local polynomial approximations. In particular, Theorem 3.7 provides a constructive, near-optimal approximation result for deep ReLU networks in Sobolev spaces. It matches—up to a logarithmic factor—the lower bounds derived from the theory of *nonlinear widths*, which measure how efficiently a class of functions can be approximated by nonlinear methods. More precisely, fix the dimension d and the smoothness s, and let p be a positive integer. If there exist continuous mappings

$$\mathcal{F} : B^{s,\infty}([0,1]^d) \to \mathbb{R}^p \quad \text{(encoder)}, \qquad \mathcal{G} : \mathbb{R}^p \to C([0,1]^d) \quad \text{(decoder)},$$

such that $\|f - \mathcal{G}(\mathcal{F}(f))\|_\infty \leq \varepsilon$ for all $f \in B^{s,\infty}([0,1]^d)$, then necessarily $p \geq c\,\varepsilon^{-d/s}$. While such results quantify how approximation rates depend on smoothness s and dimension d, they also reveal the persistence of the *curse of dimensionality* through the factor $\epsilon^{-d/s}$.

3.4 Breaking the curse of dimensionality

Discontinuous weight selection If the encoder $\mathcal{F}$ is allowed to be *discontinuous* (i.e., the weights depend non-smoothly on f), one can prove a milder lower bound $p \geq c\,\varepsilon^{-d/(2s)}$. Yarotsky showed in 2018 that ReLU networks can achieve this improved scaling, confirming their near-optimal efficiency among nonlinear approximation schemes [15].

Adaptive architecture These results assume a *fixed architecture* whose weights adapt to each target function. If one allows the network structure (depth or width) to depend adaptively on f, tighter rates can be achieved—similar to adaptive finite element methods (AFEM).

Compositional structure Standard function spaces are fundamentally constrained by their nonlinear widths. By constructing function spaces tailored to compositional structure and invoking the Kolmogorov–Arnold superposition theorem, Montanelli *et al.* [9] demonstrated that, for a subset of continuous functions, the approximation complexity can be reduced from $\mathcal{O}(\epsilon^{-d})$ to $\mathcal{O}(\epsilon^{-\log d})$.

Barron spaces A classical theorem of Carathéodory states that if a point $x \in \mathbb{R}^d$ lies in the convex hull of a set P, then x can be expressed as a convex combination of at most $d+1$ points in P.

Theorem 3.8 (Carathéodory's theorem)**.** *Let $P \subset \mathbb{R}^d$, and let $x \in \operatorname{conv}(P)$, the convex hull of P. Then there exist points $p_1, \ldots, p_{d+1} \in P$ and nonnegative coefficients $\lambda_1, \ldots, \lambda_{d+1}$ with $\sum_{i=1}^{d+1} \lambda_i = 1$ such that*

$$x = \sum_{i=1}^{d+1} \lambda_i p_i.$$

Maurey's theorem extends this result to infinite-dimensional Hilbert spaces.

Theorem 3.9 (Maurey's theorem)**.** *Let H be a Hilbert space with norm $\|\cdot\|$, and let $G \subset H$ satisfy $\|g\| \leq b$ for all $g \in G$ and some $b > 0$. Then, for every f in the convex hull of G and every integer $n \geq 1$, there exists f_n in the convex hull of n points in G such that*

$$\|f - f_n\|^2 \leq \frac{c}{n},$$

where $c > b^2 - \|f\|^2$ is a constant.

Following Barron [1], consider the class of functions $f : \mathbb{R}^d \to \mathbb{R}$ that admit a Fourier representation

$$f(\boldsymbol{x}) = \int_{\mathbb{R}^d} F(\boldsymbol{w}) e^{i\boldsymbol{w}\cdot\boldsymbol{x}} d\boldsymbol{w},$$

for some complex-valued function $F : \mathbb{R}^d \to \mathbb{C}$ such that the weighted integral

$$\|f\|_{\mathcal{B}} := \int_{\mathbb{R}^d} \|\boldsymbol{w}\|_2 |F(\boldsymbol{w})| d\boldsymbol{w} < \infty.$$

Here $\|\boldsymbol{w}\|_2 = \sqrt{w_1^2 + \cdots + w_d^2}$ is the Euclidean norm. The quantity $\|f\|_{\mathcal{B}}$ is called the *Barron norm* of f.

Theorem 3.10 (Barron 1993)**.** *Let f have finite Barron norm $\|f\|_{\mathcal{B}} < \infty$. Then, for every integer $n \geq 1$, there exists a shallow sigmoidal network $\hat{f}_n$ with n hidden neurons such that*

$$\|f - \hat{f}_n\|_{L^2([0,1]^d)} \leq \frac{\|f\|_{\mathcal{B}}}{\sqrt{n}}.$$

Equivalently, to achieve an approximation error at most ϵ, it suffices to take

$$n = \mathcal{O}\left(\frac{\|f\|_{\mathcal{B}}^2}{\epsilon^2}\right).$$

Remark 3.11. This shows that functions with Fourier transforms having finite first moment (weighted by $\|\boldsymbol{w}\|_2$) are efficiently approximated by shallow networks. Importantly, this rate is independent of the input dimension d.

Generalized bandlimited functions We can extend Barron's theory to deep ReLU networks and more general function representations [10]. Following this idea, consider functions $f : [0,1]^d \to \mathbb{R}$ of the form

$$f(\boldsymbol{x}) = \int_{[-1,1]^d} F(\boldsymbol{w}) K(\boldsymbol{w} \cdot \boldsymbol{x})\, d\boldsymbol{w},$$

where $K : \mathbb{R} \to \mathbb{C}$ is analytic and $F \in L^2([-1,1]^d)$.

Definition 3.12 (Generalized bandlimited functions)**.** The Hilbert space of generalized bandlimited functions is

$$\mathcal{H}_K([0,1]^d) = \left\{ f(\boldsymbol{x}) = \int_{[-1,1]^d} F(\boldsymbol{w}) K(\boldsymbol{w} \cdot \boldsymbol{x})\, d\boldsymbol{w} \mid F \in L^2([-1,1]^d) \right\},$$

with inner product $\langle f, g \rangle_{\mathcal{H}_K} := \int F(\boldsymbol{w}) \overline{G(\boldsymbol{w})}\, d\boldsymbol{w}$ and norm $\|f\|_{\mathcal{H}_K} := \|F\|_{L^2([-1,1]^d)}$.

Point evaluations are bounded:

$$|f(\boldsymbol{x})| \lesssim \int_{[-1,1]^d} |F(\boldsymbol{w})|\, d\boldsymbol{w} \lesssim \|f\|_{\mathcal{H}_K}.$$

Hence, $\mathcal{H}_K([0,1]^d)$ is a reproducing kernel Hilbert space containing standard bandlimited functions ($K(t) = e^{it}$).

Theorem 3.13 (Deep ReLU approximation)**.** *Let $f \in \mathcal{H}_K(B)$. Then, for any $\epsilon > 0$, there exists a deep ReLU network $\hat{f}$ with inputs in $[0,1]^d$ such that*

$$\|f - \hat{f}\|_{L^2([0,1]^d)} \leq \epsilon.$$

Moreover, the depth L and size W of $\hat{f}$ scale logarithmically and polynomially in $1/\epsilon$, respectively:

$$L = \mathcal{O}(\log^2(\|f\|_{\mathcal{H}_K}/\epsilon)), \qquad W = \mathcal{O}(\|f\|_{\mathcal{H}_K}^2/\epsilon^2 \cdot \log^2(\|f\|_{\mathcal{H}_K}/\epsilon)).$$

Remark 3.14. This extends Barron's original $O(\epsilon^{-2})$ shallow network scaling to deep networks and generalized bandlimited representations.

4 Structural equivalence with finite elements

We begin in 1D with linear elements and show that hat functions are simple linear combinations of shifted/scaled ReLUs; thus a shallow ReLU layer with fixed breakpoints parameterizes the *entire* linear Finite Element (FE) space. This immediately yields a dictionary view: weights/biases $\leftrightarrow$ scaling/nodes; width $\leftrightarrow$ local resolution.

1D case—from finite elements to ReLU networks In one dimension, consider a uniform partition

$$0 = x_0 < \cdots < x_j = jh < \cdots < x_{N+1} = 1, \qquad h = 1/(N+1),$$

and the *linear finite element space*

$$V_h = \{v \in C([0,1]) : v|_{[x_i,x_{i+1}]} \text{ linear for } i = 0,\ldots,N,\ v(0) = v(1) = 0\}.$$

The nodal ("hat") basis $\{\phi_i\}_{i=1}^N$ is

$$\phi_i(x) = \begin{cases} \dfrac{x - x_{i-1}}{h}, & x \in [x_{i-1}, x_i], \\ \dfrac{x_{i+1} - x}{h}, & x \in [x_i, x_{i+1}], \\ 0, & \text{otherwise}, \end{cases} \qquad i = 1,\ldots,N.$$

These functions are continuous, locally supported, and form a partition of unity. Their slopes $\pm 1/h$ change exactly at the mesh nodes.

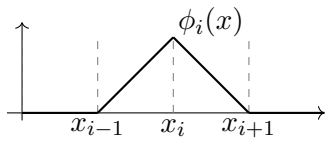


Figure 8: A 1D linear finite element hat function.

ReLU representation of the hat With $\sigma(x) = \max(0, x)$,

$$\phi_i(x) = \frac{1}{h}\Big[\sigma(x - x_{i-1}) - 2\sigma(x - x_i) + \sigma(x - x_{i+1})\Big].$$

Hence each hat is a combination of three shifted ReLUs, and

$$V_h = \operatorname{span}\{\sigma(x - x_i)\}_{i=0}^N.$$

Equivalently, a shallow ReLU network with fixed breakpoints $\{x_i\}$ parameterizes V_h:

$$v(x) = \sum_{i=0}^N a_i\, \sigma(w_h x + b_i), \qquad w_h = 1/h, \quad b_i = -x_i/h.$$

Figure 9: Left: hat basis $\{\phi_i\}$. Right: shifted ReLU ramps $\{\sigma(x - x_i)\}$. Both span the same V_h.

Dictionary view and shallow network Weights and biases play the role of mesh scaling and nodes: $w_h = 1/h$ fixes the slope within each cell, and b_i places the kink at x_i. A one-hidden-layer ReLU network with fixed breakpoints therefore parameterizes a single-level FE space. Formally, in 1D,

$$V_h = \operatorname{span}\{\sigma(x - x_i)\}_{i=0}^{N+1} = \left\{\sum_{i=0}^N a_i\, \sigma(w_h x + b_i)\right\}.$$

In multiple dimensions, shallow ReLU networks generate *continuous piecewise-linear* functions over hyperplane partitions, but depth is needed to recover finite element nodal bases when $d \geq 2$.

From shallow to deep: hierarchy and locality

- **Shallow networks** correspond to single-level FE spaces with fixed partitions. They can *approximate* locally supported nodal bases by density, but cannot reproduce them *exactly* in finite width when $d \geq 2$.
- **Deep networks** introduce a hierarchical composition of affine pieces: successive layers can implement min, max, and absolute-value operations to glue local affine regions together. This hierarchical structure mirrors multilevel finite element refinements.

The following result shows that deep ReLU networks can exactly represent any linear FE function [4, Cor. 3.1]. (We state a simplified version; see the original paper for full technical assumptions.)

Theorem 4.1 (Deep ReLU realizes linear FE)**.** *Given a regular finite element grid, any linear finite element function with N degrees of freedom can be represented by a ReLU deep neural network of depth $\mathcal{O}(d)$ and size $\mathcal{O}(d\kappa^d N)$, for some constant $\kappa \geq 2$ depending on the grid.*

In contrast, the next result shows that such a representation is impossible with only one hidden layer. Depth is not just convenient—it is *necessary*.

Theorem 4.2 (Depth is necessary in $d \geq 2$)**.** *For $d \geq 2$, not all linear FE functions can be represented exactly by a shallow (single-hidden-layer) ReLU network with finite width.*

Table 1: Structural analogies between finite elements and neural networks.

Concept	Finite Element Method	Neural Network
Local basis	hat $\phi_i(x)$	activation $\sigma(w_h x + b_i)$
Support	element $[x_{i-1}, x_{i+1}]$	region $\{x : w_h x + b_i > 0\}$
Mesh node	grid point x_i	bias $b_i = -w_h x_i$
Refinement	smaller h	more neurons (new hyperplanes)
Multilevel basis	hierarchical FE	additional layers (depth)
Order k	$\mathbb{P}_k$ elements	$\sigma_k(x) = [\max(0, x)]^k$

ReLU^k activations and higher-order elements Replacing $\sigma(x) = \max(0, x)$ by $\sigma_k(x) = [\max(0, x)]^k$ yields *piecewise polynomials* of degree k:

$$u(x) = \sum_{j=1}^{N} \alpha_j \left[\max(0, w_j^\top x + b_j)\right]^k.$$

Thus, shallow ReLU^k networks parameterize continuous piecewise-$\mathbb{P}_k$ spaces, and deeper ReLU^k networks generate richer (curved) piecewise-polynomial elements.

Takeaways

- **FEM–NN equivalence:** Deep ReLU networks *realize* linear FE spaces in any dimension $d \geq 1$, with depth scaling linearly in d.
- **1D correspondence:** In one dimension, a shallow ReLU network with fixed weights $w_h = 1/h$ and biases $b_i = -x_i/h$ exactly parameterizes the linear FE space on a uniform mesh.
- **Higher dimensions:** For $d \geq 2$, additional layers are required to recover local support; depth restores the exact realization of linear FE functions.
- **Higher-order generalization:** ReLU^k activations extend this correspondence to higher-order, piecewise-polynomial FE spaces.

5 Exercises

Exercise 5.1. Show that the set of polynomial functions satisfies the assumptions of Theorem 2.1. For shallow networks, find a counterexample.

Exercise 5.2. Show that the function H defined in Lemma 3.3 verifies

$$H\left(\frac{k}{2^n}\right) = \left(\frac{k}{2^n}\right)^2, \quad 0 \le k \le 2^n, \quad n \ge 0.$$

Exercise 5.3. Show that the ϕ_ℓ's defined in the proof of Theorem 3.7 by

$$\phi_\ell(x) = \psi\Big(3L\Big(x - \frac{\ell}{L}\Big)\Big), \quad \psi(x) = \sigma(x+2) + \sigma(x-2) - \sigma(x+1) - \sigma(x-1),$$

are a partition of unity, that is, for any $x \in [0,1]$, $\sum_{\ell=0}^{L} \phi_\ell(x) = 1$.

Exercise 5.4 (Probabilistic proof of Maurey's theorem)**.** Let H be a Hilbert space and $A \subset H$ a bounded set. Let $x \in \mathrm{conv}(A)$, and fix $n \ge 1$. The goal is to construct $x_1, \dots, x_n \in A$ such that

$$\Big\|x - \frac{1}{n}\sum_{i=1}^{n} x_i\Big\| \le \frac{c}{\sqrt{n}},$$

for some constant c depending only on A.

1. Show that since $x \in \mathrm{conv}(A)$, there exists a probability measure μ supported on A such that
$$x = \int_A y \, d\mu(y) = \mathbb{E}[Y],$$
where Y is a random variable with law μ.
2. Let $(X_i)_{i\ge1}$ be independent and identically distributed random variables with law μ. Show that
$$\mathbb{E}[X_i] = x \quad \text{for all } i.$$
3. Define the empirical average $\bar{X}_n := \frac{1}{n}\sum_{i=1}^{n} X_i$. Show that its expected squared error satisfies
$$\mathbb{E}\Big\|x - \bar{X}_n\Big\|^2 = \frac{1}{n}\mathbb{E}\|X_1 - x\|^2.$$
Hint: Use independence of the X_i and the formula $\mathrm{Var}(\frac{1}{n}\sum X_i) = \frac{1}{n^2}\sum \mathrm{Var}(X_i)$.
4. Define
$$c^2 := \mathbb{E}\|X_1 - x\|^2.$$
Argue that c is finite and depends only on the set A.
5. Conclude that there exists at least one realization ω of $(X_1, \dots, X_n)$ such that
$$\Big\|x - \frac{1}{n}\sum_{i=1}^{n} X_i(\omega)\Big\| \le \frac{c}{\sqrt{n}}.$$
Hint: Use the probabilistic method: if the expectation of a nonnegative random variable is E, then there exists an outcome where the variable is at most E.
6. Set $x_i := X_i(\omega)$ for $i = 1, \dots, n$ and show that the desired bound holds:
$$\Big\|x - \frac{1}{n}\sum_{i=1}^{n} x_i\Big\| \le \frac{c}{\sqrt{n}}.$$

6 Suggested reading

Some of this chapter's material is detailed in [3, 11]. Universality theorems are proved in [2, 5, 6], with a comprehensive review in [12]. Quantitative Sobolev estimates $W^{s,q}([0,1]^d)$ were first obtained by Yarotsky [14, 15] for $q = \infty$ and extended to all q with the optimal rate $\epsilon^{-d/(2s)}$ in [13]. Exploiting special structure—integral (Barron) [1, 10], multilevel (Korobov) [8], or compositional (Kolmogorov–Arnold) [9]—can mitigate, and in some cases overcome, the curse of dimensionality. Connections with finite elements appear in [4].

Table 2: *TL;DR table of 15 favorite references. Courses, books, and key papers grouped by topic.*

	Courses @ X	Books	Key papers
Universality theorems	–	[3]	[2, 5, 6, 11, 12]
Quantitative results	–	[3]	[11, 13, 14, 15]
Overcoming the curse	–	–	[1, 8, 9, 10]
Links with Finite Elements	–	–	[4]

References

[1] A. Barron, *Universal approximation bounds for superpositions of a sigmoidal function*, IEEE Trans. Inf. Theory, 39 (1993), pp. 930–945.

[2] G. Cybenko, *Approximation by superpositions of a sigmoidal function*, Math. Control Signals Syst., 2 (1989), pp. 303–314.

[3] B. Després, *Neural Networks and Numerical Analysis*, De Gruyter, Berlin, 2022.

[4] J. He, L. Li, J. Xu, and C. Zheng, *Relu deep neural networks and linear finite elements*, J. Comp. Math., 38 (2020), pp. 502–527.

[5] K. Hornik, M. Stinchcombe, and H. White, *Multilayer feedforward networks are universal approximators*, Neural Netw., 2 (1989), pp. 359–366.

[6] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken, *Multilayer feedforward networks with a nonpolynomial activation function can approximate any function*, Neural Netw., 6 (1993), pp. 861–867.

[7] W. S. McCulloch and W. Pitts, *A logical calculus of the ideas immanent in nervous activity*, Bull. Math. Biol., 5 (1943), pp. 115–133.

[8] H. Montanelli and Q. Du, *New error bounds for deep ReLU networks using sparse grids*, SIAM J. Math. Data Sci., 1 (2019), pp. 78–92.

[9] H. Montanelli and H. Yang, *Error bounds for deep ReLU networks using the Kolmogorov–Arnold superposition theorem*, Neural Netw., 129 (2020), pp. 1–6.

[10] H. Montanelli, H. Yang, and Q. Du, *Deep ReLU networks overcome the curse of dimensionality for generalized bandlimited functions*, J. Comp. Math., 39 (2021), pp. 801–815.

[11] P. Petersen and J. Zech, *Mathematical Theory of Deep Learning*, arXiv preprint, 2407.18384 (2024).

[12] A. Pinkus, *Approximation theory of the MLP model in neural networks*, Acta Numer., 8 (1999), pp. 143–195.

[13] J. W. Siegel, *Optimal approximation rates for deep ReLU neural networks on Sobolev and Besov spaces*, J. Mach. Learn. Res., 24 (2023), pp. 1–52.

[14] D. Yarotsky, *Error bounds for approximations with deep ReLU networks*, Neural Netw., 94 (2017), pp. 103–114.

[15] ——, *Optimal approximation of continuous functions by very deep ReLU networks*, in COLT Proceedings, 2018.